

Day 8 Lab Hour

|||||

Overview

|||||

- 实现搜索引擎和电子邮件客户端中的 Auto-complete
- 使用 Perplexity 对模型进行评估

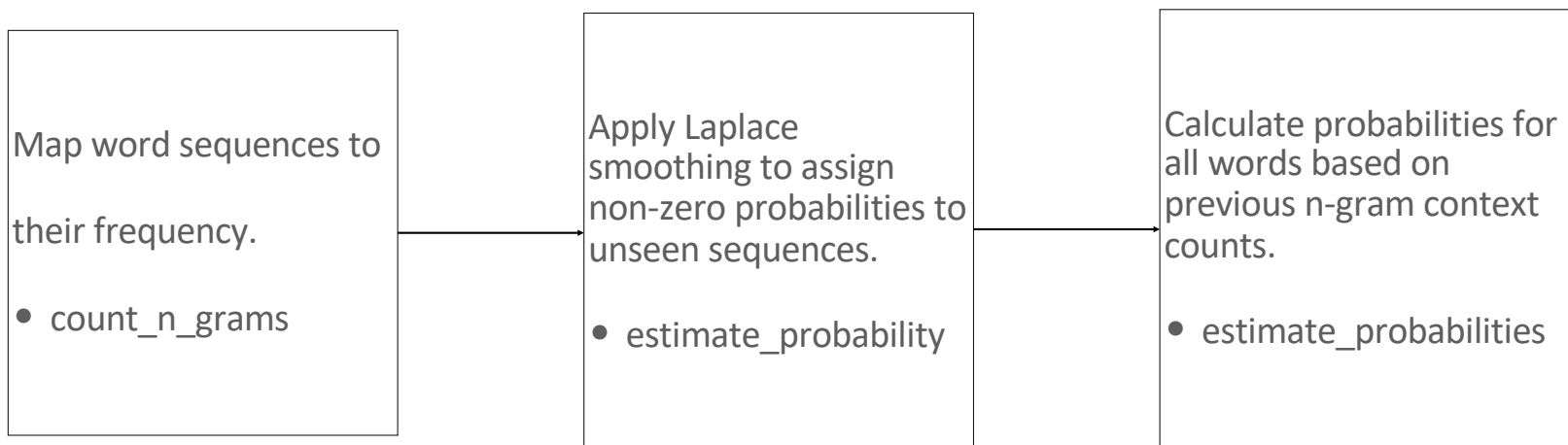
Prepare Dataset

|||||

- 实现搜索引擎和电子邮件客户端中的 Auto-complete
- Steps:
 - Load Data: 读取 en_US.twitter.txt source file
 - Tokenization: 将句子拆分为 lowercase tokens
 - Train/Test Split: 将 80% 分配给 Training, 20% 分配给 Testing
 - Preprocess Data: 将出现次数少于 N 次的稀有单词替换为 <unk>

N-Gram Models

|||||



$$\hat{P}(w_t | w_{t-1} \dots w_{t-n}) = \frac{C(w_{t-1} \dots w_{t-n}, w_t)}{C(w_{t-1} \dots w_{t-n})}$$

单词 t-1 到 t-n 在训练数据中出现后，单词 “t” 出现的次数
 单词 t-1 至 t-n 在训练数据中出现的次数

Perplexity

|||||

$$PP(W) = \sqrt[N]{\prod_{t=n}^{N-1} \frac{1}{P(w_t | w_{t-n} \cdots w_{t-1})}}$$

- N :句子的长度。
- n : n -gram 中的单词数

n -grams 告诉我们的句子信息越多，概率越高，perplexity就越低。

完成代码

```
# 前面词的数量
n = len(list(n_gram_counts.keys())[0])

# 在句子前面添加<s>，在后面添加<e>
sentence = ["<s>"] * n + sentence + ["<e>"]

# 将句子从列表转换为元组
sentence = tuple(sentence)

# 句子的长度 (包括<s>和<e>)
N = len(sentence)

# 'product_pi' 是(1/P)的累积乘积
product_pi = 1.0

### 在这条注释下方开始你的代码 ###

### 在这条注释上方完成你的代码 ###
return perplexity
```